# Guía del notebook `3_1_capa_convolucional`: la operación de una capa convolucional

**Puntos:** 4 · **Tiempo:** 30 min · **Módulo:** puente a redes profundas (clase 7) · **Archivo:** `notebooks/3_1_capa_convolucional.ipynb`

## ¿Qué vas a aprender?

- Qué es una **imagen** para la computadora: una matriz de números.
- Qué es un **kernel** (filtro) y cómo se **desliza** sobre la imagen: la correlación cruzada.
- A programar a mano la operación central de las **redes convolucionales** (CNN), que se usan para reconocer imágenes.

## Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/3_1_capa_convolucional.ipynb` → kernel **nni**.
   El notebook abre `files/Lenna.png`, así que debe abrirse desde la carpeta del repo.
3. Ejecuta las celdas en orden con **Shift + Enter**.

## La idea en 2 minutos

### La imagen

Una imagen en color son 3 matrices: rojo, verde y azul. Aquí se usa solo el **canal rojo**, una matriz de **256 × 256** con valores de 0 (negro) a 255 (blanco).

### El kernel y la correlación cruzada

Un **kernel** es una matriz pequeña, aquí de 3 × 3. Se coloca sobre cada píxel de la imagen y se calcula:

$G[i, j] = \Sigma_u \Sigma_v H[u, v] \cdot I[i + u, j + v]$

En palabras:

1. Pon el kernel centrado en el píxel (i, j).
2. Multiplica cada número del kernel por el píxel que tiene debajo.
3. Suma los 9 productos.
4. Ese total es el píxel (i, j) de la imagen de salida G.

- Los índices u y v van de $-k$ a k. Con un kernel de 3 × 3, k = 1, y van de −1 a 1.
- En la matriz de Python, el elemento H[u, v] de la fórmula está en `H[u + k, v + k]`, porque los índices de Python empiezan en 0.

### El kernel de este notebook: detector de bordes verticales

```
[[-1/3  0  1/3]
 [-1/3  0  1/3]
 [-1/3  0  1/3]]
```

Resta la columna de la izquierda y suma la de la derecha. Por eso da un número **grande y positivo** donde la imagen pasa de **oscuro a claro** de izquierda a derecha (un borde vertical), y **negativo** en el caso contrario.

### Bordes de la imagen

En las orillas el kernel "se sale" de la imagen. Con **padding 0** (sin relleno), esos píxeles no se calculan. La salida es más pequeña: 256 − 2k = **254 × 254**.

## Paso a paso

### Celdas 1–2: paquetes y la imagen

✅ **Comprobación:** `Dimensiones de la imagen: (256, 256)` y `Tipo de datos de la matriz: uint8`.

### Celda del kernel

Construye el detector de bordes y lo muestra.

### Celda `TODO`: las dos funciones (3 puntos)

**`correlacionPixel(H, I, i, j)` (1 punto)**

- Con dos `for` anidados: `u` y `v` en `range(-k, k + 1)`. Acumula en `sumatoria` el producto `H[u + k, v + k] * I[i + u, j + v]`.
- Al final, `resultado` debe ser un número de NumPy, porque la función hace `resultado.astype(int)`. Por ejemplo, `resultado = np.array(sumatoria)`.
- Atajo opcional: `np.sum(H * I[i-k:i+k+1, j-k:j+k+1])` hace lo mismo en una línea.

**`correlacionCruzadaValida(H, I)`**

- *(1 punto)* `G` debe ser una matriz de ceros **del mismo tamaño** que I: `np.zeros((m, n))`.
- *(1 punto)* Al final, quita las orillas que no se llenaron. Pista: recorta `k` filas y `k` columnas de cada lado, `G[k:m-k, k:n-k]`.

### Celda "Prueba"

Aplica tu función, luego la ReLU (que deja en 0 los negativos), y dibuja tres imágenes.

✅ **Comprobación:**

- la imagen "Correlación cruzada" se ve gris, con los contornos marcados en claro y oscuro;
- "ReLU aplicada" se ve negra, con **solo algunos bordes verticales** en blanco;
- debe parecerse a la figura `files/ccnout.png` del notebook;
- el resultado mide **254 × 254**: compruébalo con `resultado.shape`.

### Pregunta (1 punto)

**¿Por qué tras la ReLU solo se ven algunos bordes verticales?**

- El kernel da valores **positivos** en los bordes donde se pasa de **oscuro (izquierda) a claro (derecha)**, y **negativos** en los bordes de claro a oscuro.
- La ReLU convierte todos los negativos en 0, así que esos bordes desaparecen.
- Tampoco se ven los bordes **horizontales**, porque este kernel solo compara izquierda contra derecha.

## Para pensar

- Si **volteas** el kernel (los +1/3 a la izquierda), ¿qué bordes aparecen? → Los otros, los que antes eran negativos.
- En una CNN, los números del kernel **no se eligen a mano**: la red los aprende. Eso es el notebook `3_2`.

## Errores frecuentes

| Síntoma | Causa | Solución |
|---|---|---|
| `AttributeError: 'float' object has no attribute 'astype'` | `resultado` es un número normal de Python | `resultado = np.array(sumatoria)` |
| `IndexError: index 256 is out of bounds` | Recorriste hasta el borde | Los `for` deben ir de `k` a `m-k` (ya vienen así) |
| La imagen sale con un marco negro | No recortaste G | `return G[k:m-k, k:n-k]` |
| Valores raros, enormes o negativos de golpe | Operaste con enteros `uint8`, que "dan la vuelta" al pasar de 255 | Multiplica por el kernel, que es float, o convierte con `I.astype(float)` |
| `FileNotFoundError: files/Lenna.png` | Notebook abierto fuera de la carpeta | Abre la carpeta `curso-redes-neuronales` |
| Tarda unos segundos | Son 254 × 254 píxeles × 9 productos con `for` | Es normal |